45. API Güvenliği & Rate Limiting: Sistemini Korumak

Brute-Force ve DoS Saldırılarına Karşı Hız Sınırları ve Güvenlik Başlıkları
Bölüm 8 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Bir saldırgan `/api/login` endpoint'ine saniyede 10.000 şifre denemesi gönderdiğinde sunucunuz neden 20 saniyede çöker ve Rate Limiting sistemi nasıl kurtarır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Rate Limiting (Hız Sınırlaması), bir IP veya kullanıcının belirli bir zaman diliminde yapabileceği istek sayısını sınırlayan emniyet sübabıdır. - 📌 Helmet.js, tarayıcıları koruyan kritik HTTP güvenlik başlıklarını (Content-Security-Policy, HSTS) tek satırda enjekte eder. - 📌 CORS (Cross-Origin Resource Sharing), rastgele yabancı web sitelerinin kullanıcılarınız adına API istekleri atmasını engeller.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir gece kulübünün kapısını düşünün.

Kapıda bir güvenlik görevlisi (Bouncer) var. Eğer güvenlik olmasaydı ne olurdu? 5.000 kişilik bir holigan grubu aynı anda kapıya yüklenir, kapıyı kırar ve kulübü yerle bir ederdi! (DDoS Saldırısı).

Güvenlik görevlisi ne yapar? - Kapıda sıra oluşturur: *"Her dakikada sadece 5 kişi içeri girebilir!"* (Rate Limiting). - Girenlerin üzerini arar, kesici aletleri toplar (Input Sanitization). - Davetiyesi olmayanları kapıdan çevirir (CORS).

Yazılımda bir API geliştirdiğinizde internetteki tüm botlar ve kötü niyetli tarayıcılar o kapıya koşar. Kapıya bir nöbetçi koymazsanız sunucunuz 1 dakikada çöker.

🧠 2. AI Çağında Yeni Zihniyet Modeli

API Güvenlik Duvarı Mimarisi:

```
[Gelen İstek: POST /api/login]
              |
              ▼
[1. HELMET: Güvenlik Başlıkları] (HSTS, No-Sniff, CSP)
              |
              ▼
[2. CORS KONTROLÜ] ──> "İstek izinli domainden (app.sirket.com) mi geliyor?"
              |
              ▼
[3. RATE LIMITER (Redis)] ──> "Bu IP son 1 dakikada 5'ten fazla denedi mi?"
              |
      ┌───────┴──────┐
      ▼              ▼
[LİMİT AŞILDI (429)] [ONAYLANDI (Geçebilir)]
(Too Many Requests)    |
                       ▼
            [4. ZOD DOĞRULAMASI]
            (Zararlı karakterleri filtrele)
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: API Güvenliği ve Altyapı Uzmanı
Express.js projemize kapsamlı bir güvenlik zırhı ekleyeceğiz.

## GÖREV:
1. `express-rate-limit` kullanarak `/api/auth/*` rotalarına dakikada en fazla 5
deneme sınırı koyan ve 429 Too Many Requests dönen middleware'i yaz.
2. `helmet` ve katı `cors` yapılandırmasını güvenli şekilde kur.
3. Redis tabanlı bir rate limiter'ın (Sliding Window) neden çoklu sunucularda
(Cluster) zorunlu olduğunu açıkla.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
// Kötü / Güvensiz Express Kurulumu: CORS her yere açık, hız sınırı yok, güvenlik başlığı yok!
import express from 'express';
const app = express();

app.use(cors()); // Tehlikeli: Tüm siteler (saldırganlar dahil) API'ye erişebilir!
// Rate limit yok: Saldırgan saniyede 100.000 şifre deneyebilir!
app.post('/login', (req, res) => { /* ... */ });
```

✅ Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):

```
import express from "express";
import helmet from "helmet";
import cors from "cors";
import rateLimit from "express-rate-limit";

const app = express();

// 1. Güvenlik Başlıkları (Helmet)
app.use(helmet());

// 2. Katı CORS Yapılandırması
const allowedOrigins = ["https://app.sirketimiz.com"];
app.use(cors({
  origin: (origin, callback) => {
    if (!origin || allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error("CORS İlkesi İhlali: Bu domaine erişim izni yok."));
    }
  },
  credentials: true, // HttpOnly çerezlere izin ver
  methods: ["GET", "POST", "PUT", "DELETE"]
}));

// 3. Hassas Giriş Rotaları İçin Hız Sınırı (Rate Limiting)
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 Dakika
  max: 5, // IP başına maksimum 5 başarısız deneme
  standardHeaders: true, // RateLimit-* standart başlıklarını dön
  legacyHeaders: false,
  message: {
    status: 429,
    error: "Çok fazla başarısız giriş denemesi. Lütfen 15 dakika sonra tekrar deneyin."
  }
});

// Giriş rotasına limitleyiciyi bağla
app.use("/api/auth/login", authLimiter);
```

💡 **Kod Analizi:** `helmet` ile kritik HTTP güvenlik başlıkları eklenmiş, `cors` ile sadece resmi şirket domainine izin verilmiş ve `authLimiter` ile şifre kaba kuvvet (Brute-Force) saldırıları 5 denemede durdurulmuştur.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Çoklu sunucu (cluster / Docker) ortamında yerel bellek (in-memory) rate limiter kullanmak.
  - **Çözüm:** Sunucu sayısı birden fazlaysa rate limiting durumunu merkezi bir Redis kümesinde tutun.
- **Tuzak:** CORS ayarında `origin: '*'` bırakıp kimlik doğrulamalı istekleri savunmasız bırakmak.
  - **Çözüm:** Origin olarak daima açıkça onaylanmış güvenilir frontend alan adlarını belirtin.
- **Tuzak:** DDoS saldırılarını sadece uygulama katmanında çözmeye çalışmak.
  - **Çözüm:** Büyük hacimli volumetrik saldırılar için Cloudflare veya AWS Shield gibi CDN katmanlarını kullanın.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📹 Ders Başlığı: *Ders 45: API Güvenliği - Rate Limiting ve Helmet ile Korunma*

- 🖥️ **Slayt 1:** Brute-Force ve DoS saldırılarının maliyeti.
- 🖥️ **Slayt 2:** Rate Limiting mantığı: Sliding Window vs Token Bucket.
- 🖥️ **Slayt 3:** Helmet.js ile tek satırda 11 güvenlik başlığı enjekte etmek.
- 🖥️ **Slayt 4:** CORS (Cross-Origin Resource Sharing) nedir ve nasıl yapılandırılır?

Canlı Kodlama / Ekran Kaydı Senaryosu:
Express API'de login rotasına rate limit koyun. Bir for döngüsüyle arka arkaya 6 kez istek atın. İlk 5 isteğin 200 aldığını, 6. isteğin anında `429 Too Many Requests` ile engellendiğini canlı terminalde gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizdeki API'ye `helmet` paketini ekleyin ve `/login` rotasına IP başına 15 dakikada en fazla 10 istek sınırı koyan bir rate limiter bağlayın.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir API endpoint'ine belirlenen limitin üzerinde istek yapıldığında dönülmesi gereken standart HTTP durum kodu hangisidir? - A) 200 OK - B) 404 Not Found - C) 429 Too Many Requests - D) 500 Internal Server Error > **Doğru Yanıt:** C > **Açıklama:** `429 Too Many Requests` durum kodu, istemcinin hız limitini aştığını ve belirli bir süre beklemesi gerektiğini bildiren RFC standardıdır.

Soru 2: CORS (Cross-Origin Resource Sharing) politikasının temel görevi nedir? - A) İnterneti hızlandırmak - B) Bir web sitesinde çalışan zararlı JavaScript kodunun, kullanıcının bilgisi olmadan başka bir domaindeki API'ye yetkisiz istekler göndermesini tarayıcı seviyesinde engellemek - C) Veritabanını yedeklemek - D) CSS kodlarını küçültmek > **Doğru Yanıt:** B > **Açıklama:** CORS, tarayıcıların aynı köken politikasını (Same-Origin Policy) esnetirken hangi yabancı domainlerin API'ye erişebileceğini denetleyen güvenlik kalkanıdır.